Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

**High-Performance Computing Lab for CSE** **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

# Report for Project 4

Due date: 27 April 2026, 18:00 (Europe/Zurich)

**HPC Lab for CSE 2025 — Submission Instructions**

**Following these instructions is mandatory**: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

# 1. Computing $\pi$ with MPI [10 points]

## 1.1. Parellization using MPI

The parellel implementation of the $\pi$ computation is based on the idea of dividing the interval $[0, 1]$ into $N$ subintervals, where $N$ is the number of processes. Each process computes the contribution to $\pi$ from its assigned subinterval and then all contributions are summed together using MPI. In listing 1 we show the implementation of the parallel $\pi$ computation using MPI.

Listing 1: Parallel $\pi$ computation using MPI

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

h = 1.0 / (double)N;

MPI_Barrier(MPI_COMM_WORLD);
time_start = walltime();

for (uint64_t i = (uint64_t)rank; i < N; i += (uint64_t)size) {
    double x = (i + 0.5) * h;
    local_sum += 4.0 / (1.0 + x * x);
}

MPI_Reduce(&local_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
double time = walltime() - time_start;
if (rank == 0) {
    pi = sum * h;
    printf("# pi N procs time [secs]\n");
    printf("%.15f %20" PRIu64 " %3d %.8f\n", pi, N, size, time);
    }

MPI_Finalize();
```

Each process starts from its rank and iterates over the indices of the subintervals assigned to it by incrementing the index by the total number of processes in each iteration. This way, each process computes the contribution to $\pi$ from its assigned subintervals without overlap. After the local sums are computed, they are reduced to a single sum using `MPI_Reduce`, which is then used to compute the final value of $\pi$ on the root process (rank 0).

## 1.2. Strong scaling

To evaluate the strong scaling of the parallel $\pi$ computation, we run the program with a fixed problem size $N$ and varying number of processes. The results are shown in.

## 1.3. Weak scaling

## 2. Ring sum using MPI [10 points]

In listing 2 we show the implementation of the ring sum using MPI.

Listing 2: Ring sum using MPI

```
int size, rank;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

// IMPLEMENT: Ring sum algorithm
int sum = rank;
for (int i = 0; i < size - 1; i++) {
  int send_to = (rank + 1) % size;
  int recv_from = (rank - 1 + size) % size;

  int recv_sum;
  MPI_Sendrecv(&sum, 1, MPI_INT, send_to, 0,
               &recv_sum, 1, MPI_INT, recv_from, 0,
               MPI_COMM_WORLD, MPI_STATUS_IGNORE);

  sum += recv_sum;
}
printf("Process %i: Sum = %i\n", rank, sum);

// Finalize MPI
MPI_Finalize();
```

The implementation uses a **ring communication pattern**, where each process exchanges data only with its immediate neighbors in a cyclic topology. Specifically, process $i$ sends its current partial sum to process $(i+1) \bmod P$ and receives a value from process $(i-1+P) \bmod P$, where $P$ is the total number of processes.

Each process initializes its local sum with its rank and performs $P-1$ communication steps. At every iteration, it forwards its current partial sum to the next process and accumulates the value received from the previous one. This ensures that, after all iterations, every process has received contributions from all other processes and therefore holds the global sum.

To avoid deadlock issues, the implementation uses the `MPI_Sendrecv` routine instead of separate blocking send and receive calls. This function performs the send and receive operations in a single, coordinated step, ensuring that communication progresses safely without requiring explicit ordering between processes.

## 3. Cartesian domain decomposition and ghost cells exchange [20 points]

## 4. Parallelizing the Mandelbrot set using MPI [30 points]

## 5. Parallel matrix-vector multiplication and the power method [30 points]